

Exercises Associations and ArrayList

For all exercises:

- Use the start folders on Canvas
- Create the class(es) in the package *model* according to the given UML Diagram. No more and no less. **You are not allowed to create additional or other attributes or methods than the one you find in the UML Diagram...**
- After you have written these class(es), run the tests
- If all tests are successful, create objects of the class in the main method of your application and experiment with the different methods. Don't forget to import the class you have created...

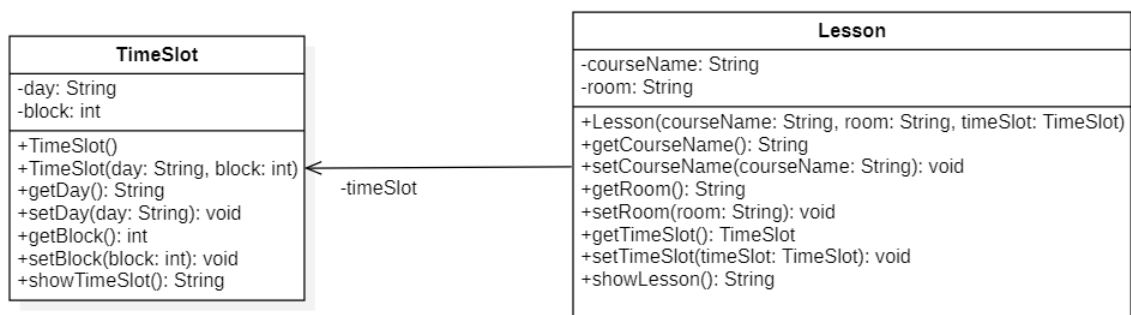
Exercise 1

A student's timetable consists of a number of objects of the Lesson type.

Examples:

Java	Monday 5	P108Z
English	Wednesday 3	G112

Create the classes **TimeSlot** and **Lesson** according to the following UML Diagram:



1. In the class **TimeSlot**
 - The no-arg constructor sets the day to 'Monday' and the block to 1.
 - The method *showTimeSlot()* returns the next string:
e.g. Wednesday, block 4
2. In the class **Lesson**
 - The method *showLesson* returns the following string (use the method *showTimeSlot()* of TimeSlot):
e.g. Friday, block 2: Java in room G204
3. In **ExerciseLessonApplication**
 - Create an object of the class Lesson with:
 - course name = IOT Essentials
 - room = P108Z
 - timeSlot = Thursday block 3
 - Show all data of this lesson using the method *showLesson()*:

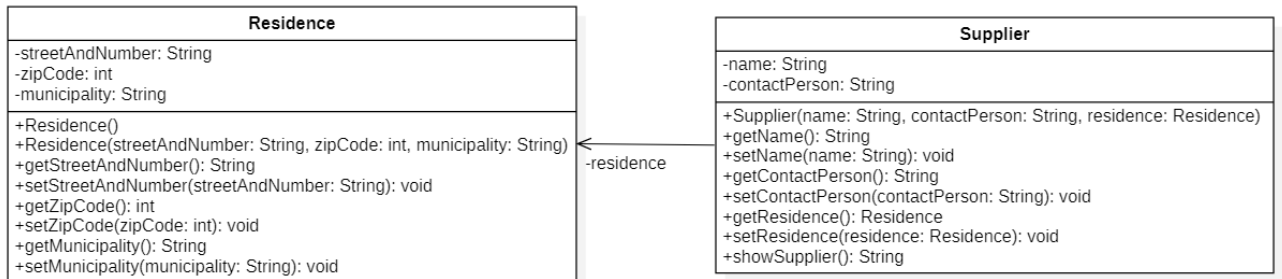
Thursday, block 3: IOT Essentials in room P108Z

- Show the following data of the lesson using the getters.

Course name: IOT Essentials
Where: P108Z
When: Thursday, block 3

Exercise 2

Create the classes **Residence** and **Supplier** according to the following UML Diagram:



1. In the class **Residence**

- the no-arg constructor sets the place of residence at the following address:
 - streetAndNumber = "no data"
 - zipCode = 0
 - municipality = "no data"

2. In the class **Supplier**

- the method *showSupplier* returns the following string:
Supplier "U&Me", contact person Jef Perkens
Address: Berkendreef 24 bus 7, 2000 Antwerpen

NB: You can show the double quote by using the escape character just before this quote: \. You can insert a line break by using the escape character + n: \n.

3. In **ExerciseSupplierApplication**

- Create a Supplier object with the following data:
 - name = U&Me
 - contact person = Jef Perkens
 - residence = Berkendreef 24 bus 7, 2000 Antwerpen
- Show all data of the supplier by using the method *showSupplier()*

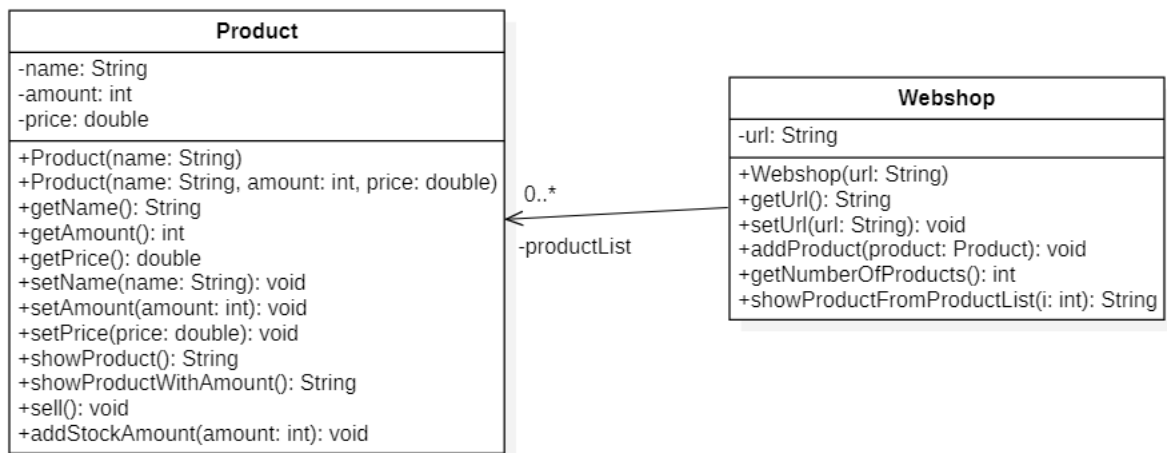
```
Supplier "U&Me", contact person Jef Perkens
Address: Berkendreef 24 bus 7, 2000 Antwerpen
```

- Show the following details of the supplier by using the getters

```
Municipality of supplier U&Me: Berkendreef 24 bus 7, Antwerpen
Contact person: Jef Perkens
```

Exercise 3

Create the classes **Product** and **Webshop** according to the following UML Diagram:



a) In the class **Product**:

- *showProduct* returns the following result:
E.g. The product Candle costs €6.73
- *showProductWithAmount* returns the following result:
E.g. The product Candle (10 pieces in stock) costs €6.73
- *sell* reduces the number of pieces by 1. Of course you should check if there is stock of that product. If there is no stock left, it will remain at 0. In other words, the stock cannot become negative.
- *addStockAmount* is used to change the stock of a product. Please note: if the parameter is negative, you may not change the stock!

b) In the class **Webshop**:

- *addProduct* adds a Product object to the productList
- *getNumberOfProducts* returns the number of elements in the productList
- *showProductFromProductList* returns:
 - a string with the product that can be found in the list at the position indicated by the parameter *i*
e.g. The product Keyring (30 pieces in stock) costs €6.7
 - when the parameter indicates a non-existing position in the list, the returnvalue is
no product with this index in the list

c) In **ExerciseProductApplication**:

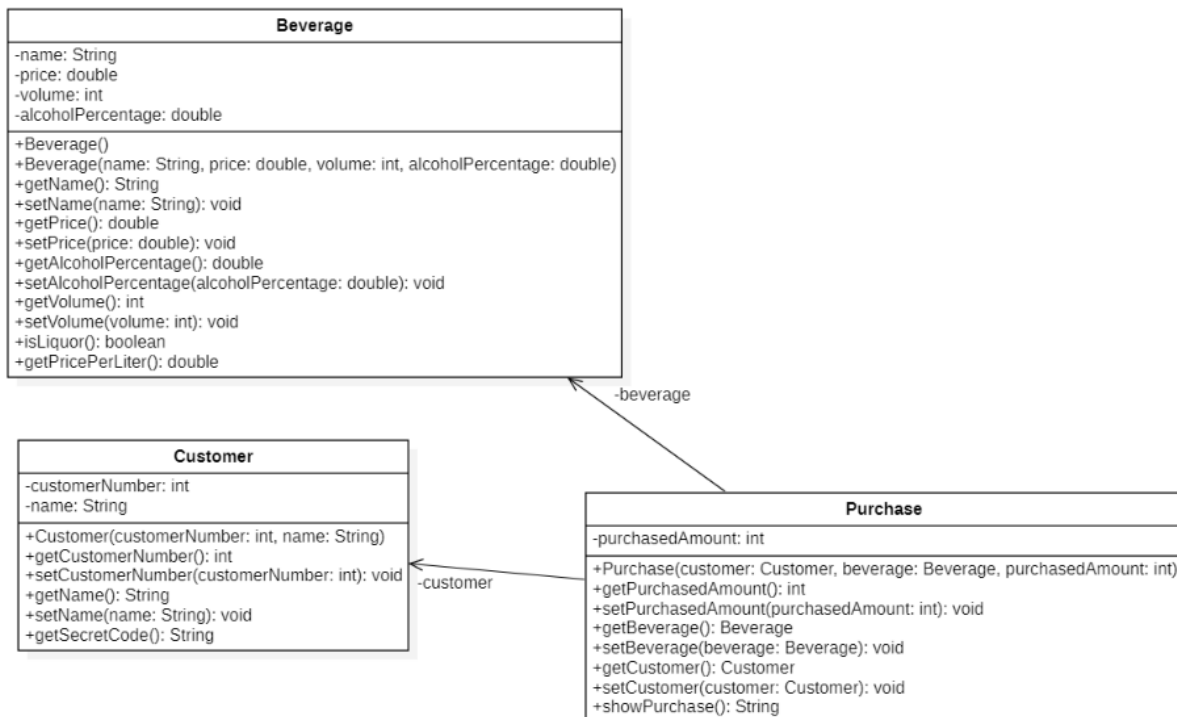
- Create 5 products.

- Create a webshop, choose a name for that shop and add the 5 products to the shop.
- Display the webshop with all the products on the screen.

```
Welcome to my webshop www.nicethingstohave.com
The product Candle (10 pieces in stock) costs €6.73
The product Necklace (8 pieces in stock) costs €46.35
The product Bracelet (25 pieces in stock) costs €13.2
The product Keyring (30 pieces in stock) costs €6.7
The product Dreamcatcher (100 pieces in stock) costs €4.77
```

Exercise 4

Create the classes **Beverage**, **Customer** and **Purchase** according to the following UML Diagram:



1. In the **Beverage** class
 - the method *isLiquor* returns true when the alcohol percentage is at least 35. Otherwise the return value is false.
 - the method *getPricePerLiter* returns the price per liter. For example: you have a 500 ml bottle that costs €10. The method will then answer that this drink costs €20 per liter.
2. In the **Customer** class
 - the method *getSecretCode* returns a String which is composed as follows: the first 2 letters of the name + the customer number + the last 2 letters of the name and put it all in capital letters.

For example for customer *Mestdagh* with number *175*, the return value is *ME175GH*

3. In the **Purchase** class

- the method *showPurchase()* returns the following string:

Customer Vandergoten bought 3 bottle(s) of whiskey (11.5%)

4. In **ExerciseBeverageApplication**:

- Create a Purchase object
customer: Peter Vergoten with customer number 123
beverage: Famous Grouse whiskey, price:14.69, volume:1000,
alcohol%:38.5
amount purchased: 5
- Show all the details of this object using the method *showPurchase()*

```
Customer Peter Vergoten bought 5 bottle(s) of Famous Grouse whiskey (38.5%)
```

- Then show the following data using the getters. The text "is a liquor" or "is not a liquor" depends on the return value of the method *isLiquor()*.

```
Customer: Peter Vergoten has secret code PE123EN  
Beverage: Famous Grouse whiskey with alcohol percentage 38.5  
This beverage is a liquor and costs 14.69 euro per liter  
Amount purchased: 5
```

- Create a second object of the Purchase class with following data:
Same customer: Peter Vergoten with customer no. 123
beverage: Jupiler, price:0.5, volume:250, alcohol%: 4.2
amount purchased: 50
- Show all data of this object by using the right methods:

```
Customer Peter Vergoten bought 50 bottle(s) of Jupiler (4.2%)  
Customer: Peter Vergoten has secret code PE123EN  
Beverage: Jupiler with alcohol percentage 4.2  
This beverage is not a liquor and costs 2.0 euro per liter  
Amount purchased: 50
```